

Data Analytics (DA)- Project II

Analyzing E-Learning Platform Purchases using MySQL Script

```
/* 1. Database Setup & Data Entry */
```

```
Create database Elearning_platforms ;
```

```
Use Elearning_platforms ;
```

```
Create table learners (  
  learner_id Int primary key auto_increment ,  
  Full_name varchar (100),  
  Country varchar(100 )  
);
```

```
create table Courses (  
  course_id Int Primary key ,  
  course_name varchar(100),  
  Category varchar(50) ,  
  unit_price decimal (10,2)  
);
```

```
Create table Purchases (  
  purchase_id Int primary key ,  
  learner_id int ,  
  course_id int ,  
  quantity int ,  
  purchase_date date ,
```

```
Foreign key (Learner_id)
references learners(learner_id )
On delete cascade
on update cascade ,
```

```
Foreign key (course_id)
References courses(course_id)
On delete cascade
On update cascade
) ;
```

Insert into learners values

```
(1,'Arun Kumar','India'),
(2,'Priya Sharma','India'),
(3,'John Smith','USA'),
(4,'Emma Wilson','UK'),
(5,'Rahul Verma','India');
```

Insert into courses values

```
(101,'SQL for Beginners','Beginner',3000),
(102,'Python Programming','Programming',5000),
(103,'Power BI Dashboard','Analytics',7000),
(104,'Advanced SQL','Intermediate',6000),
(105,'Excel for Data Analysis','Analytics',4000);
```

Insert into purchases values

```
(1,1,101,2,'2025-01-10'),
(2,1,103,1,'2025-01-15'),
(3,2,102,1,'2025-02-01'),
(4,3,104,2,'2025-02-05'),
```

```
(5,4,105,3,'2025-02-10'),  
(6,5,101,1,'2025-02-15'),  
(7,5,102,2,'2025-02-20'),  
(8,2,103,1,'2025-03-01');
```

2. Data Exploration Using Joins

```
/* Combine learner, course, and purchase data */
```

-- Inner Join

Select

```
    l.Full_name as Learners_name ,  
    c.course_name as Course_name ,  
    c.Category ,  
    p.quantity as Quantity ,  
    format(p.quantity * c.unit_price ,2 ) as Total_Amount ,  
    purchase_date as Purchase_Date  
from purchases as p  
inner join learners as l  
on l.learner_id = p.learner_id  
inner join courses as c  
on c.course_id = p.course_id  
order by Total_Amount desc ;
```

-- Left Join

Select

```
    l.full_name,
```

```

c.course_name ,
p.quantity ,
format(p.quantity * c.unit_price ,2 ) as Total_Amount
from learners as l
left join purchases as p
on l.learner_id = p.learner_id
left join courses as c
on p.course_id = c.course_id
order by Total_Amount desc
;

```

-- Right Join

```

select
    l.full_name ,
    c.course_name ,
    p.quantity ,
    format ( p.quantity * c.unit_price ,2) as Total_Amount
from courses as c
right join purchases as P
on c.course_id = p.course_id
left join learners as l
on l.learner_id = p.learner_id
order by Total_Amount desc ;

```

/*3. Core Analytical Queries

-- Q1. Display each learner's total spending with their country.*/

Select

```

    l.Full_name as Learners_Name,

```

```

l.country as Country,
Format(Sum( p.quantity *c.unit_price),2 )as Total_Spend
from learners as l
    join purchases as p
on l.learner_id = p.learner_id
join courses as c
on c.course_id = p.course_id
group by l.learner_id ,l.full_name ,l.country
;

```

-- Q2. Find the top 3 most purchased courses by quantity.

```

select
    c.course_name,
    sum(p.quantity) as Total_Purchased
from purchases as p
join courses as c
on p.course_id = c.course_id
group by c.course_id ,c.course_name
order by Total_purchased desc
limit 3 ;

```

-- Q3. Show each category's Total revenue & Number of unique learners

```

Select
    c.category ,
    format(Sum(p.quantity * c.unit_price ),2) as Total_Revenue ,
    count(distinct p.learner_id ) as unique_learners

```

```
from purchases as p
join courses as c
on c.course_id = p.course_id
group by c.category
;
```

-- Q4. List learners who purchased from more than one category.

```
Select
    l.full_name,
    count(distinct c.category) as category_count
from purchases as p
join learners as l
on l.learner_id = p.learner_id
join courses as c
on c.course_id = p.course_id
group by l.full_name
having category_count >1
;
```

-- Q5. Identify courses never purchased.

```
select
    c.course_id ,c.course_name
from courses as c
left join purchases as p
on c.course_id = p.course_id
where p.purchase_id is null ;
```

/* 4. Subqueries & Correlated Subqueries */

-- Q6. Find learners whose total spending is above the average learner spending.

```
select
    l.full_name
  from learners as l
 where learner_id in (
select
    l.learner_id
  from purchases as p
 join courses as c
 on p.courseid = c.course_id
 group by learner_id
 having Total_spend > (
select
avg(total_Spending) as Avg_Spending
from (
Select
    sum(p.quantity * c.unit_price ) as Total_Spending
  from purchases as p
 join courses as c
 on p.courseid = c.course_id
 group by learner_id )
)
) ;
```

-- Q7. Display courses whose price is higher than any course in the 'Beginner' category.

```

select
    course_name ,Unit_price
from courses
where unit_price > any (
    select
        unit_price
    from courses
    where category = "Beginner" );

```

-- Q8 . Find learners who spent more than the average spending in their country.

```

SELECT
    l.full_name,
    l.country
FROM learners l
JOIN
(
    SELECT
        l.learner_id,
        l.country,
        SUM(p.quantity*c.unit_price) AS spending
    FROM learners l
    JOIN purchases p
    ON l.learner_id=p.learner_id
    JOIN courses c
    ON p.course_id=c.course_id
    GROUP BY l.learner_id,l.country
) s
ON l.learner_id=s.learner_id

```



```

WHERE spending >
(
SELECT
    AVG(s2.spending)
FROM
    (
SELECT
        12.learner_id,
        12.country,
        SUM(p2.quantity*c2.unit_price) AS spending
    FROM learners 12
    JOIN purchases p2
    ON 12.learner_id=p2.learner_id
    JOIN courses c2
    ON p2.course_id=c2.course_id
    GROUP BY 12.learner_id,12.country
    ) s2
WHERE s2.country=s.country
);

```

/* 5. CTE, CASE, View, and NULL Handling */

-- Q9. Use a CTE to calculate total spending per learner, then:
-- Display learners with spending above 10,000.

```

With Learner_spending as (
select
    1.full_name,
    sum(quantity * Unit_price) as Total_Spending
from learners as 1

```

```

join purchases as p
on l.learner_id = p.learner_id
join courses as c
on c.course_id = p.course_id
group by l.full_name
)
select *
    from learner_Spending
where
    Total_Spending > 10000 ;

```

/* Q10. CASE Expression

Classify learners based on spending:

- Above 15,000 → “High Value”,
- 8,000-15,000 → “Medium Value”,
- Below 8,000 → “Low Value” */

```

select
    l.full_name ,
    sum(p.quantity * c.unit_price ) as Total_Spending,
Case
    when sum(p.quantity * c.unit_price ) > 15000
then 'High Value'
    when sum(p.quantity * c.unit_price ) between 8000 and 15000
then 'Medium Value'
    else 'Low Value'
end as Customer_Category

from learners as l

```

```

join purchases as p
ON l.learner_id=p.learner_id
Join courses c
ON p.course_id=c.course_id
group by l.learner_id ,l.full_name ;

```

/* Q11 . NULL Handling

**Display all courses and replace NULL purchase counts with 0 using:
IFNULL() or**

COALESCE() */

```

select
    c.course_name,
    coalesce(sum(p.quantity * c.unit_price ),0 ) as purchase_count
from courses as c
left join purchases as p
on c.course_id = p.course_id
group by c.course_id , c.course_name ;

```

/* Q12 . View

**• Create a view: category_performance_view • Showing • Category •
Total revenue • Number of purchases • Average revenue per purchase
*/**

Create view Category_Performance_view as

```

select
    c.category,
    sum(p.quantity * c.unit_price ) as Total_Revenue,
    count(p.purchase_id) as Number_of_Purchases ,
    avg(p.quantity * c.unit_price ) as Average_revenue_per_purchase
from courses as c

```

```
join purchases as p  
on c.course_id =p.course_id  
group by c.category ;
```

```
select * from Category_performance_view ;
```
